Q3 - HashSet and the hashCode-equals Contract

SIMATS Online Programming Lab - JAVA PROGRAMMING - Assignment 2 (CO2)

Bloom's Level: K4 - Analyse | 6 Marks

Name	Sanjay Adithya S J	Register No.	192311336
Java version	OpenJDK 21.0.10	GitHub	github.com/sanoryx/CSA0921-JAVA

1. Complete the Code - run before and after TODO 1

The program de-duplicates gate scans of students by registration number. **Note:** the screenshot imports only `java.util.Set`, so `new HashSet<>()` would not compile; `import java.util.HashSet;` was added (the Q3 instructions list it as the first line). The main class is placed first only so the single-file launcher can run it; the logic is unchanged.

Step 1 - before TODO 1 (equals() overridden, hashCode() not overridden):

```
public class AttendanceBefore {
    public static void main(String[] args) {
        Set<Student> present = new HashSet<>();
        present.add(new Student("192311001", "Arun"));   // Gate 1
        present.add(new Student("192311002", "Divya"));
        present.add(new Student("192311001", "Arun"));   // Gate 3
        System.out.println("Unique students: " + present.size());
    }
}
// Student: equals() compares regNo only; hashCode() is NOT overridden.
```

```
Recorded output (3 consecutive runs, identical):
Unique students: 3
```

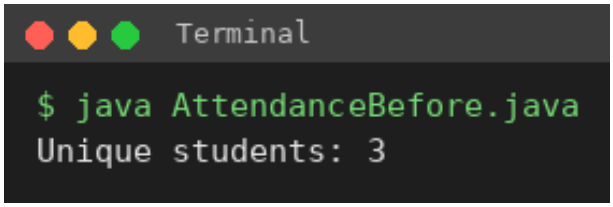


Figure 1 - Console output of AttendanceBefore (rendered from the recorded run, OpenJDK 21.0.10).

Step 2 - after TODO 1 (hashCode() added):

```
@Override
public int hashCode() {
    return regNo.hashCode();   // uses exactly the field that equals() uses
}
```

```
Recorded output (3 consecutive runs, identical):
Unique students: 2
```

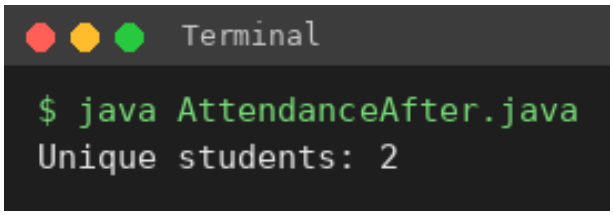


Figure 2 - Console output of AttendanceAfter (rendered from the recorded run, OpenJDK 21.0.10).

Explanation of the difference. Without hashCode(), the inherited `Object.hashCode()` returns an identity-based value, so the two `Student("192311001", ...)` objects normally hash to different values even though `equals()` says they are equal. HashSet compares hash codes first and calls equals() only on entries with the same hash, so the second Arun is never compared with the first and is stored as a new element (size 3). Once hashCode() is derived from regNo, both Arun objects land in the same bucket, equals() returns true, and the second add() is rejected (size 2).

2. Designed Tests - predictions made before execution

Each prediction was written before the test was run. All results below are from `Experiments.java` (raw output in `experiments_output.txt`).

#	Test design	Prediction and reason	Observed
1	**Appears to work.** The *same* object reference is scanned twice; class has equals() only.	Size 2. Same reference gives the same identity hash, same bucket, and equals() returns true immediately through `this == o`. The missing hashCode() is hidden.	**2** (as predicted)
2	**Exposes duplicates.** Two *different* objects with the same regNo; equals() only.	Size 3. The identity hashes differ, so equals() is never consulted and the duplicate is stored.	**3** (as predicted); x.equals(y)=true but hashCodes differ
2b	Test 2 repeated with the corrected class (equals + hashCode).	Size 2. Equal objects now produce equal hashes, so they meet in one bucket.	**2** (as predicted)
3	**Own test: mutate regNo after insertion.** Class has hashCode() and equals() but regNo is non-final. Add object m, then change m.regNo.	size() stays 1. contains(m) becomes false because the hash is now computed from the new regNo and points to the wrong bucket. Adding a new equal object then gives size 2 (a logical duplicate).	After mutation: size=1, contains(m)=**false**, contains(old regNo)=**false**. After adding an equal object: size=**2**, set shows [192311999, 192311999].
4	**Own test: TreeSet whose comparator disagrees with equals().** Students 192311001/Arun and 192311002/Arun; TreeSet compared by name only.	TreeSet size 1 (comparator says equal) while HashSet size 2 (equals says different).	HashSet=**2**, TreeSet(by name)=**1**. A TreeSet by regNo gives 2, as does HashMap keyed by regNo.

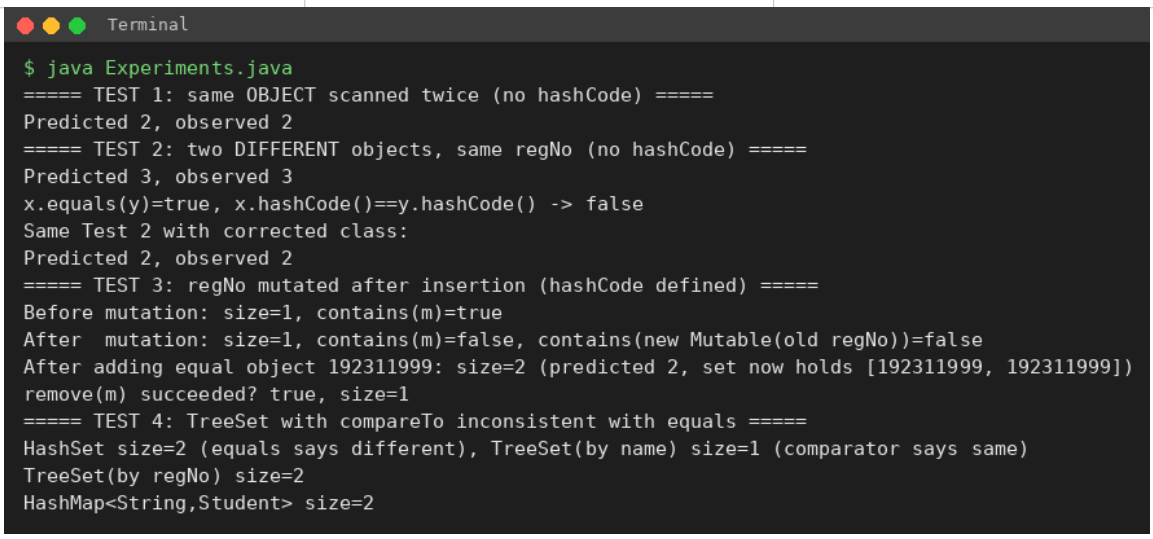


Figure 3 - Console output of Experiments.java, tests 1-4 (rendered from the recorded run).

Every prediction matched the observation. The important lesson from Tests 1 and 2 is that a faulty class can pass a casual test (Test 1) and still fail in production (Test 2); the bug only shows up when two *distinct* objects represent the same logical student, which is exactly what two gate scanners create.

```
Raw output extract:
===== TEST 1 =====  Predicted 2, observed 2
===== TEST 2 =====  Predicted 3, observed 3 ; x.equals(y)=true, hashCode equal? false
                    corrected class: Predicted 2, observed 2
===== TEST 3 =====  Before mutation: size=1, contains(m)=true
                    After  mutation: size=1, contains(m)=false,
                    contains(new Mutable(old regNo))=false
                    After adding equal object: size=2  [192311999, 192311999]
                    remove(m) succeeded? true
===== TEST 4 =====  HashSet size=2, TreeSet(by name) size=1, TreeSet(by regNo)=2, HashMap size=2
```

3. Inquiry

3.1 Why the duplicates appear

HashSet is backed by a HashMap. It finds an element in two steps: (1) compute `hashCode()` and map it to a bucket; (2) inside that bucket, test candidates with `equals()`. Duplicates appear when two logically equal objects produce different hash codes, because step 1 sends them to different buckets (or the stored hash differs) and step 2 never runs between them.

3.2 How HashSet uses hashCode() and equals() to locate an element

Step	Action
1	h = hashCode() of the key; the table index is derived from h (after a bit-spreading step).
2	Go to that bucket (linked list, or a tree if the bucket is large).
3	For each node in the bucket: if node.hash == h **and** (node.key == key **or** key.equals(node.key)), the element is found.
4	If found, add() returns false and the set is unchanged; if not found, a new node is added (size + 1).

3.3 What happens when a field used in hashCode() is changed after insertion

The entry stays in the bucket chosen by the *old* hash, but lookups now use the *new* hash. In Test 3, contains(m) returned false even though m is physically in the set, searching with the old regNo also failed because equals() now compares against the new value, and a logically equal object could be added (size 2). The element is effectively lost: it still counts in size() and appears when iterating, but contains() and, in general, remove() can no longer find it. (In the run above, remove(m) returned true only because it removed the *newly added* equal object, whose stored hash matches the current hash; the stale entry for m stayed in the set, so size() was still 1 afterwards.) This is why keys in hash-based collections should be immutable; the Student class declares regNo as `final`.

3.4 How to correct the Student class and how to verify the correction

```
class Student {
    private final String regNo;      // final: hash cannot change after insertion
    private final String name;
    Student(String regNo, String name) { this.regNo = regNo; this.name = name; }

    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Student)) return false;
        return regNo.equals(((Student) o).regNo);
    }
    @Override public int hashCode() { return regNo.hashCode(); }  // same field as equals()
    @Override public String toString() { return regNo + " " + name; }
}
```

Verification: (a) the program prints 2 instead of 3; (b) Test 2b gives 2 for distinct objects with the same regNo; (c) `a.equals(b)` implies `a.hashCode() == b.hashCode()` for equal pairs; (d) the reverse is not required, because two different students may share a hash code (a collision) and equals() resolves it; (e) name is deliberately left out of both methods, because two scans of the same student must match even if the name is typed differently.

4. The equals-hashCode Contract (reasoned explanation)

Rule	Meaning	Consequence if broken
Consistency over time	While no field used by equals() changes, repeated hashCode() calls return the same int.	Mutated key is lost in the set (Test 3).
Equal objects, equal hashes	If a.equals(b) is true, then a.hashCode() == b.hashCode().	Duplicates in HashSet and duplicate keys in HashMap (Test 2).
Unequal objects (not required)	If a.equals(b) is false, hash codes may be equal or different; different hashes improve performance.	Only affects speed: constant hashCode() is legal but turns every lookup into a scan of one bucket.

The contract is not arbitrary: HashSet makes the hash code a *necessary condition* for equality (a fast filter). If equal objects could have different hashes, the filter would reject true matches. Therefore hashCode() must be a function of a subset of the fields used by equals(), never of anything else. Here both use regNo.

5. Alternative Concept and Complexity Analysis

5.1 De-duplication with TreeSet or HashMap

```
// TreeSet with a Comparator on regNo - no hashCode() needed
Set<Student> present = new TreeSet<>(Comparator.comparing(Student::getRegNo));
present.add(s);                       // add() returns false if comparator returns 0

// HashMap keyed by regNo - the key is a String, which already has a correct hashCode()
Map<String, Student> present = new HashMap<>();
present.putIfAbsent(s.getRegNo(), s); // keeps the first scan
```

TreeSet identifies duplicates through the comparator (`compare(a,b) == 0`), not equals()/hashCode(), so the faulty-hashCode problem cannot occur; it requires the comparator to be consistent with equals (Test 4 shows what

happens otherwise). HashMap sidesteps the contract by delegating hashing to the String key; the Student class needs no hashCode(), although the map is then keyed on a value the caller must extract each time.

5.2 Complexity

Structure	Time per scan (add/lookup)	Space for n unique students	Notes
HashSet<Student>	O(1) average; O(log n) worst case in Java 8+ (treeified bucket), O(n) with a constant hashCode on older JDKs	O(n): one node per element plus the table	Unordered. Needs correct equals() + hashCode().
TreeSet<Student> (Comparator on regNo)	O(log n) always (comparisons along a red-black tree path)	O(n): one tree entry per element, no table	Sorted by regNo. Needs a comparator consistent with equals.
HashMap<String, Student>	O(1) average, as for HashSet	O(n): same node cost as HashSet; extra only if the key String is not shared with the Student	Can also store extra data per regNo (e.g. first scan time).

Space estimate (64-bit HotSpot, compressed references; computed from object layout, not measured): a HashMap node is 12-byte header + hash + key + value + next = 32 bytes, plus 4 bytes per table slot (about 5 to 11 bytes per element at the default load factor), roughly 37 to 43 bytes per entry; HashSet uses the same node because its map stores a shared dummy value. A TreeMap entry has a header, key, value, left, right, parent and colour flag, which is about 40 bytes, with no table. A direct measurement of heap usage for HashMap<String,Student> gave 42.5 bytes per entry, consistent with the estimate; the equivalent measurements for HashSet and TreeSet were unreliable (garbage-collector noise made them negative), so they are not used as evidence. The Student objects themselves are common to all three and are excluded.

5.3 Measured time per scan

Setup: n unique registration numbers, each scanned twice (2n scans, of which half are duplicates). 6 warm-up runs, then 11 timed runs; the **median** of the 11 runs is reported (individual runs are in the raw data). SerialGC, 3 GB heap, OpenJDK 21 on the sandbox machine. TreeSet uses String natural order, which equals a regNo comparator.

n	HashSet (ns/scan)	HashMap (ns/scan)	TreeSet (ns/scan)	TreeSet / HashSet
1,000	89.8	52.9	351.6	3.9x
10,000	35.1	27.8	324.9	9.3x
100,000	60.3	46.9	215.3	3.6x
1,000,000	34.7	24.8	244.3	7.0x

Reading the evidence honestly. TreeSet is consistently several times slower than the hash-based structures at every size, which agrees with the O(1) versus O(log n) prediction in *direction*. However the measured HashSet and HashMap times do not fall on a clean line and TreeSet's time does not grow visibly with log n here (the 1,000-element run is higher mainly because of JIT compilation and the cost of the first allocations; the per-element cost also depends on cache behaviour once the structure exceeds the CPU cache). With a range of only 1,000 to 1,000,000, log2(n) goes from about 10 to 20, and this is less than the noise between sizes. The safest conclusion is therefore: hash structures were about 4x to 9x faster per scan in this experiment, and the exact ratio should not be over-interpreted. For a campus attendance system, either choice is far faster than the physical scan rate; choose TreeSet only if a sorted listing by regNo is also needed.

6. Conclusion

Without `hashCode()` the original program reports 3 unique students; with hashCode() derived from regNo it reports the expected 2. The contract (equal objects must have equal hash codes, and the hash must stay stable while the object is in a set) is what allows HashSet to find an element without comparing it to every other element. Test 1 shows why the bug is easy to miss, Test 2 shows it, Test 3 shows what happens when a key is mutable, and Test 4 shows the same class of mistake for ordered sets. TreeSet and HashMap provide alternative de-duplication designs, with O(log n) versus O(1) average time per scan and linear space in all cases.

Appendix A - Raw Benchmark Data (ns per scan, 11 timed runs after 6 warm-up runs)

n	HashSet runs	TreeSet runs	HashMap runs
1,000	147 108 77 2108 89 77 78 80 3945 90 92	365 386 359 350 354 344 355 344 352 336 336	89 53 52 1659 50 53 54 54 52 54 52
10,000	39 32 34 34 33 35 42 36 79 45 33	353 371 322 319 125 348 321 320 340 355 325	29 26 28 28 229 55 27 30 25 67 26
100,000	50 51 53 62 80 60 54 74 61 71 48	215 219 205 339 268 254 177 208 246 212 196	38 37 49 47 58 56 36 55 53 38 47
1,000,000	22 135 35 29 40 39 42 28 29 31 35	225 242 272 244 265 278 256 209 243 222 246	25 26 24 29 31 24 25 23 27 24 21

The occasional large values (for example 2108 or 3945 ns at n = 1,000) are single runs disturbed by JIT compilation or a garbage-collection pause; this is the reason medians, not means, are reported in section 5.3.

Appendix B - AttendanceBefore.java (run before TODO 1)

```
import java.util.HashSet;
import java.util.Set;

public class AttendanceBefore {
    public static void main(String[] args) {
        Set<Student> present = new HashSet<>();

        present.add(new Student("192311001", "Arun")); // scanned at Gate 1
        present.add(new Student("192311002", "Divya"));
        present.add(new Student("192311001", "Arun")); // scanned again at Gate 3

        System.out.println("Unique students: " + present.size());
    }
}

class Student {
    private final String regNo;
    private final String name;

    Student(String regNo, String name) {
        this.regNo = regNo;
        this.name = name;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Student)) return false;
        return regNo.equals(((Student) o).regNo);
    }

    // TODO 1: hashCode() NOT yet overridden

    @Override
    public String toString() {
        return regNo + " " + name;
    }
}
```

Appendix C - AttendanceAfter.java (run after TODO 1)

```
import java.util.HashSet;
import java.util.Set;

public class AttendanceAfter {
    public static void main(String[] args) {
        Set<Student> present = new HashSet<>();

        present.add(new Student("192311001", "Arun")); // scanned at Gate 1
        present.add(new Student("192311002", "Divya"));
        present.add(new Student("192311001", "Arun")); // scanned again at Gate 3

        System.out.println("Unique students: " + present.size());
    }
}

class Student {
    private final String regNo;
    private final String name;

    Student(String regNo, String name) {
        this.regNo = regNo;
        this.name = name;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Student)) return false;
        return regNo.equals(((Student) o).regNo);
```

```
    }

    // TODO 1 (completed): hashCode() consistent with equals()
    @Override
    public int hashCode() {
        return regNo.hashCode();
    }

    @Override
    public String toString() {
        return regNo + " " + name;
    }
}
```

Appendix D - Experiments.java (tests 1-4, benchmark)

```
import java.util.*;

public class Experiments {

    // ---------- Test classes ----------
    static class NoHash {                       // equals only (the faulty class)
        final String regNo; NoHash(String r){regNo=r;}
        @Override public boolean equals(Object o){ return o instanceof NoHash n && regNo.equals(n.regNo); }
    }
    static class Fixed {                        // equals + hashCode (corrected class)
        final String regNo; Fixed(String r){regNo=r;}
        @Override public boolean equals(Object o){ return o instanceof Fixed n && regNo.equals(n.regNo); }
        @Override public int hashCode(){ return regNo.hashCode(); }
    }
    static class Mutable {                      // hashCode uses a NON-final field
        String regNo; Mutable(String r){regNo=r;}
        @Override public boolean equals(Object o){ return o instanceof Mutable n && regNo.equals(n.regNo); }
        @Override public int hashCode(){ return regNo.hashCode(); }
        @Override public String toString(){ return regNo; }
    }
    record Stu(String regNo, String name) {}    // for TreeSet test (record equals uses both fields)

    public static void main(String[] args) {
        System.out.println("===== TEST 1: same OBJECT scanned twice (no hashCode) =====");
        NoHash a = new NoHash("192311001"), b = new NoHash("192311002");
        Set<NoHash> s1 = new HashSet<>();
        s1.add(a); s1.add(b); s1.add(a);
        System.out.println("Predicted 2, observed " + s1.size());

        System.out.println("===== TEST 2: two DIFFERENT objects, same regNo (no hashCode) =====");
        Set<NoHash> s2 = new HashSet<>();
        s2.add(new NoHash("192311001")); s2.add(new NoHash("192311002")); s2.add(new NoHash("192311001"));
        System.out.println("Predicted 3, observed " + s2.size());
        NoHash x = new NoHash("192311001"), y = new NoHash("192311001");
        System.out.println("x.equals(y)=" + x.equals(y) + ", x.hashCode()==y.hashCode() -> " + (x.hashCode()==y.hashCode()));
        System.out.println("Same Test 2 with corrected class:");
        Set<Fixed> s2f = new HashSet<>();
        s2f.add(new Fixed("192311001")); s2f.add(new Fixed("192311002")); s2f.add(new Fixed("192311001"));
        System.out.println("Predicted 2, observed " + s2f.size());

        System.out.println("===== TEST 3: regNo mutated after insertion (hashCode defined) =====");
        Mutable m = new Mutable("192311001");
        Set<Mutable> s3 = new HashSet<>();
        s3.add(m);
        System.out.println("Before mutation: size=" + s3.size() + ", contains(m)=" + s3.contains(m));
        m.regNo = "192311999";
        System.out.println("After  mutation: size=" + s3.size() + ", contains(m)=" + s3.contains(m)
            + ", contains(new Mutable(old regNo))=" + s3.contains(new Mutable("192311001")));
        s3.add(new Mutable("192311999"));
        System.out.println("After adding equal object 192311999: size=" + s3.size() + " (predicted 2, set now holds " + s3 + ")");
        System.out.println("remove(m) succeeded? " + s3.remove(m) + ", size=" + s3.size());

        System.out.println("===== TEST 4: TreeSet with compareTo inconsistent with equals =====");
        Comparator<Stu> byName = Comparator.comparing(Stu::name);
        Set<Stu> ts = new TreeSet<>(byName);
        Set<Stu> hs = new HashSet<>();
        for (Stu st : List.of(new Stu("192311001","Arun"), new Stu("192311002","Arun"))) { ts.add(st); hs.add(st); }
        System.out.println("HashSet size=" + hs.size() + " (equals says different), TreeSet(by name) size=" + ts.size() + " (comparator s
        Set<Stu> ts2 = new TreeSet<>(Comparator.comparing(Stu::regNo));
        ts2.add(new Stu("192311001","Arun")); ts2.add(new Stu("192311002","Divya")); ts2.add(new Stu("192311001","Arun"));
        System.out.println("TreeSet(by regNo) size=" + ts2.size());
        Map<String,Stu> hm = new HashMap<>();
        hm.put("192311001", new Stu("192311001","Arun")); hm.put("192311002", new Stu("192311002","Divya")); hm.put("192311001", new Stu(
        System.out.println("HashMap<String,Student> size=" + hm.size());

        // ---------- Benchmark ----------
        System.out.println("===== BENCHMARK: ns per scan (each of n students scanned twice => 2n scans) =====");
        System.out.println("n,HashSet_median_ns,TreeSet_median_ns,HashMap_median_ns,all_runs(ns)");
        int[] sizes = {1_000, 10_000, 100_000, 1_000_000};
        int RUNS = 11; int WARM = 6;
        for (int n : sizes) {
            String[] ids = new String[n];
            for (int i = 0; i < n; i++) ids[i] = String.valueOf(192_000_000 + i);
            double[] hsT = new double[RUNS], tsT = new double[RUNS], hmT = new double[RUNS];
            for (int r = 0; r < RUNS + WARM; r++) {   // first WARM runs are warm-up
                long t0 = System.nanoTime();
                Set<Fixed> hset = new HashSet<>();
```

```
                for (int k = 0; k < 2; k++) for (String id : ids) hset.add(new Fixed(id));
                long t1 = System.nanoTime();
                Set<String> tset = new TreeSet<>();   // regNo comparator (natural String order)
                for (int k = 0; k < 2; k++) for (String id : ids) tset.add(id);
                long t2 = System.nanoTime();
                Map<String,String> hmap = new HashMap<>();
                for (int k = 0; k < 2; k++) for (String id : ids) hmap.putIfAbsent(id, id);
                long t3 = System.nanoTime();
                if (hset.size()!=n || tset.size()!=n || hmap.size()!=n) throw new IllegalStateException();
                if (r >= WARM) { hsT[r-WARM]=(t1-t0)/(2.0*n); tsT[r-WARM]=(t2-t1)/(2.0*n); hmT[r-WARM]=(t3-t2)/(2.0*n); }
            }
            System.out.printf("%d,%.1f,%.1f,%.1f,HS%s TS%s HM%s%n", n, med(hsT), med(tsT), med(hmT),
                fmt(hsT), fmt(tsT), fmt(hmT));
        }

        System.out.println("===== MEMORY (approx. heap after GC, bytes per unique student, n=200000, entry only) =====");
        int n = 200_000; String[] ids = new String[n];
        for (int i = 0; i < n; i++) ids[i] = String.valueOf(192_000_000 + i);
        Student[] studs = new Student[n];
        for (int i = 0; i < n; i++) studs[i] = new Student(ids[i]);
        System.out.printf("HashSet<Student>        : %.1f B/student%n", mem(() -> { Set<Student> s=new HashSet<>(); for (Student t: studs
        System.out.printf("TreeSet<Student>(cmp)   : %.1f B/student%n", mem(() -> { Set<Student> s=new TreeSet<>(Comparator.comparing((St
        System.out.printf("HashMap<String,Student> : %.1f B/student%n", mem(() -> { Map<String,Student> s=new HashMap<>(); for (Student t
    }

    static class Student { final String regNo; Student(String r){regNo=r;}
        @Override public boolean equals(Object o){ return o instanceof Student n && regNo.equals(n.regNo); }
        @Override public int hashCode(){ return regNo.hashCode(); } }

    static double med(double[] a){ double[] c=a.clone(); Arrays.sort(c); return c[c.length/2]; }
    static double avg(double[] a){ double s=0; for(double d:a) s+=d; return s/a.length; }
    static String fmt(double[] a){ StringBuilder sb=new StringBuilder("["); for(int i=0;i<a.length;i++) sb.append(String.format("%.0f%s",
    static double mem1(java.util.function.Supplier<Object> f, int n){
        System.gc(); try{Thread.sleep(200);}catch(Exception e){}
        long before = used();
        Object keep = f.get();
        System.gc(); try{Thread.sleep(200);}catch(Exception e){}
        long after = used();
        if (keep.hashCode()==42) System.out.print("");
        return (after-before)/(double)n;
    }
    static double mem(java.util.function.Supplier<Object> f, int n){ double[] v=new double[5]; for(int i=0;i<5;i++) v[i]=mem1(f,n); Array
    }
    static long used(){ Runtime r=Runtime.getRuntime(); return r.totalMemory()-r.freeMemory(); }
}
```

Appendix E - experiments_output.txt (complete console output)

```
===== TEST 1: same OBJECT scanned twice (no hashCode) =====
Predicted 2, observed 2
===== TEST 2: two DIFFERENT objects, same regNo (no hashCode) =====
Predicted 3, observed 3
x.equals(y)=true, x.hashCode()==y.hashCode() -> false
Same Test 2 with corrected class:
Predicted 2, observed 2
===== TEST 3: regNo mutated after insertion (hashCode defined) =====
Before mutation: size=1, contains(m)=true
After  mutation: size=1, contains(m)=false, contains(new Mutable(old regNo))=false
After adding equal object 192311999: size=2 (predicted 2, set now holds [192311999, 192311999])
remove(m) succeeded? true, size=1
===== TEST 4: TreeSet with compareTo inconsistent with equals =====
HashSet size=2 (equals says different), TreeSet(by name) size=1 (comparator says same)
TreeSet(by regNo) size=2
HashMap<String,Student> size=2
===== BENCHMARK: ns per scan (each of n students scanned twice => 2n scans) =====
n,HashSet_median_ns,TreeSet_median_ns,HashMap_median_ns,all_runs(ns)
1000,89.8,351.6,52.9,HS[147 108 77 2108 89 77 78 80 3945 90 92] TS[365 386 359 350 354 344 355 344 352 336 336] HM[89
    53 52 1659 50 53 54 54 52 54 52]
10000,35.1,324.9,27.8,HS[39 32 34 34 33 35 42 36 79 45 33] TS[353 371 322 319 125 348 321 320 340 355 325] HM[29 26 28
    28 229 55 27 30 25 67 26]
100000,60.3,215.3,46.9,HS[50 51 53 62 80 60 54 74 61 71 48] TS[215 219 205 339 268 254 177 208 246 212 196] HM[38 37
    49 47 58 56 36 55 53 38 47]
1000000,34.7,244.3,24.8,HS[22 135 35 29 40 39 42 28 29 31 35] TS[225 242 272 244 265 278 256 209 243 222 246] HM[25 26
    24 29 31 24 25 23 27 24 21]
===== MEMORY (approx. heap after GC, bytes per unique student, n=200000, entry only) =====
HashSet<Student>        : -41.6 B/student
TreeSet<Student>(cmp)   : -40.0 B/student
HashMap<String,Student> : 42.5 B/student
```